

CredResolve AI Voice Agent — System Design Document

This document provides a comprehensive overview of the design, architecture, and implementation details for the CredResolve AI Inbound Voice Agent.

1. Problem Statement

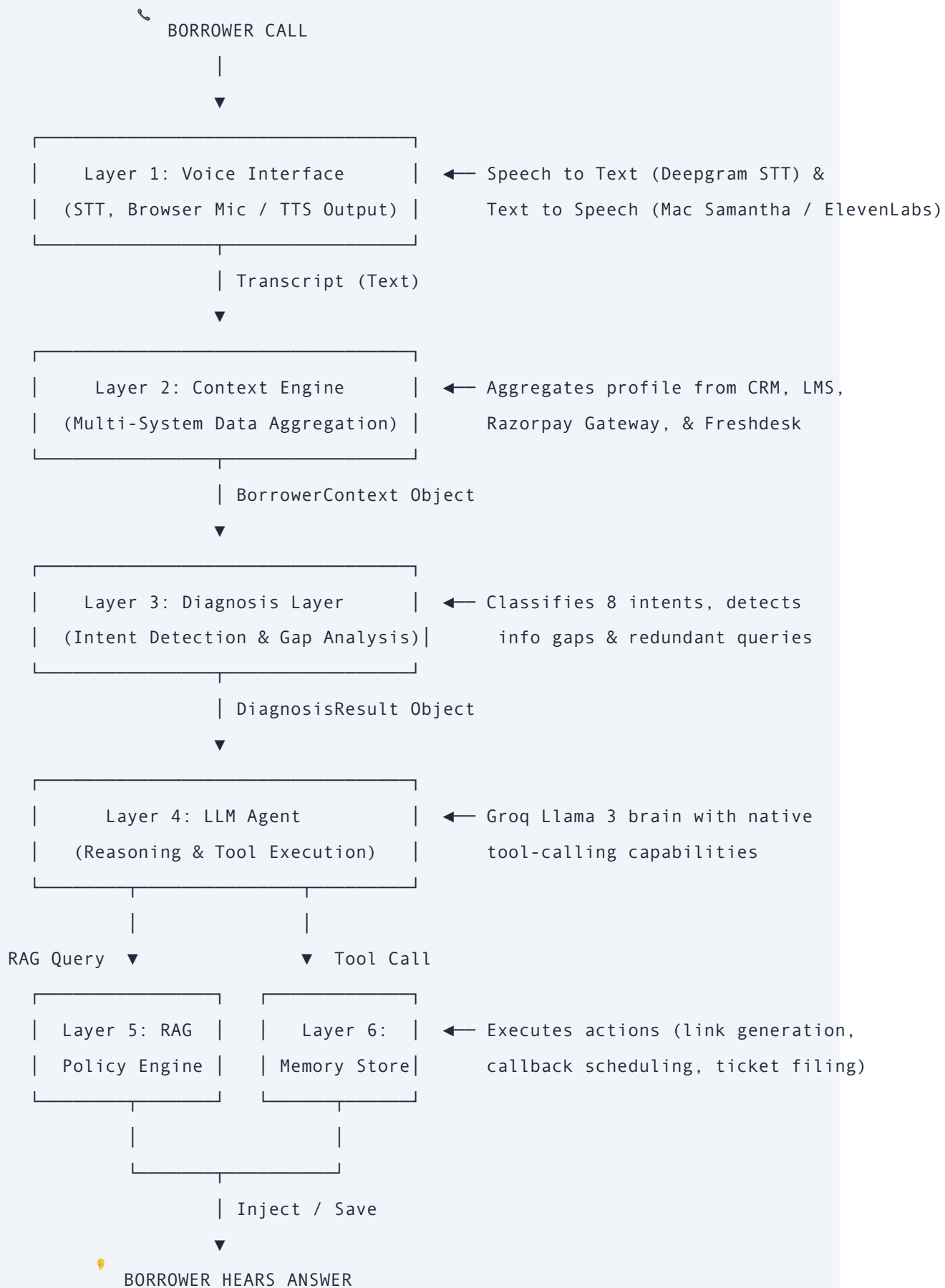
Lending institutions and financial services companies receive thousands of inbound calls from borrowers daily. These calls typically revolve around: * Inquiring about current loan status, Equated Monthly Installment (EMI) amounts, and remaining tenures. * Asking about accrued penalties, late fees, and reasons behind them. * Discussing payment gateway failures, transaction declines, or double debits. * Requesting waivers for penalties or seeking one-time settlement (OTS) paths due to financial distress. * Committing to future payment dates (promise-to-pay commitments).

Scaling customer support to handle these volumes introduces several challenges: 1. **Operational Cost:** Running 24/7 call centers with human agents is highly expensive and difficult to scale dynamically. 2. **Inconsistency:** Human agents may apply policy guidelines (like penalty waivers and settlement terms) inconsistently, leading to compliance risks. 3. **Information Fragmentation:** Customer data is scattered across CRMs, Loan Management Systems (LMS), Payment Gateways, and support tickets, making it hard for agents to get a unified view quickly. 4. **Lack of Cross-Call Context:** If a borrower calls back to follow up on a payment promise, the new agent often lacks access to previous conversation logs or promises, forcing the borrower to repeat themselves.

CredResolve addresses these challenges by providing an agentic inbound voice system that combines speech recognition, intent diagnosis, policy guidelines lookup, memory store, and automated platform integrations. The goal is an AI agent that handles these calls intelligently by understanding borrower context, diagnosing intent, retrieving policies, and learning from previous interactions.

2. System Architecture

The system is built as a modular, 6-layer architecture designed to process speech input, orchestrate reasoning, invoke enterprise systems, and persist interactions.



Layer Descriptions

1. **Layer 1: Voice Interface:** Captures microphone input from the user (via browser Web Speech API), sends audio to Deepgram STT for transcription, passes text to the agent, and synthesizes agent responses back into speech using native Mac TTS (`say -v Samantha`) or ElevenLabs.
 2. **Layer 2: Context Engine:** Runs automatically before the agent responds. Aggregates data from mock CRM, LMS, Payment Gateway, and support ticketers into a unified `BorrowerContext` object.
 3. **Layer 3: Diagnosis Layer:** Analyzes the borrower's input, cross-references it with context, and identifies the core intent, known facts, missing parameters, and next follow-up query.
 4. **Layer 4: LLM Agent:** Groq's Llama 3 model handles the conversational loop, decides which tools to execute, and generates context-grounded responses.
 5. **Layer 5: RAG Engine:** Performs keyword-based TF-IDF matches across 20 JSON policy files. Snippets are injected into the agent's system prompt.
 6. **Layer 6: Memory Store:** A SQLite-backed persistent database (`borrower_memory` and `agent_memory` tables) that stores borrower commitments (amount, date, reason), call counts, callback dates, and user preferences.
-

3. Component Design

3.1 Context Engine

The Context Engine builds a comprehensive borrower context. * **Purpose:** Aggregate customer information from disjointed platforms at the start of a call. * **Inputs:** Phone number or unique identifier. * **Outputs:** `BorrowerContext` object. * **5 Data Sources Aggregated:**

1. **Zoho CRM (Mock):** Customer profile details, KYC status, PAN details, and email.
2. **Loan Management System (LMS - Mock):** Current active loan type, loan amount, interest rate, tenure, EMI amount, outstanding principal, next due date, and delinquency status.
3. **Razorpay Gateway (Mock):** Customer transaction records, payment statuses (success, failure), gateway failure codes, payment channel, and total principal/interest paid.
4. **Freshdesk Support (Mock):** Support tickets filed, categories, status (OPEN, RESOLVED), priorities, and resolution notes.
5. **SQLite Memory Store:** Previous interaction logs, commitments, and callbacks.

* **Risk Signal Logic:** Automatically scans the profile to compute 10 flags: 1. `delinquency_status`: Repayment status of

the account (CURRENT, OVERDUE_15, OVERDUE_30, OVERDUE_60, or NPA). 2. REPEATED_FAILURES: Triggered if failed or missed payments ≥ 3 . 3. RECENT_FAILURE_STREAK: Triggered if ≥ 2 payments failed in the last 3 attempts. 4. OPEN_TICKETS: Active support tickets in Freshdesk. 5. PENDING_WAIVER_REQUEST: Open support tickets seeking late-fee waivers. 6. SETTLEMENT_IN_PROGRESS: Open tickets seeking one-time settlement. 7. BROKEN_PROMISE_TO_PAY: Commitments where fulfilled is false and due date has passed. 8. ACTIVE_PROMISE_TO_PAY: Active payment promises with due dates in the future. 9. FRUSTRATED_BORROWER: Borrower sentiment history shows negative/frustrated traits. 10. HIGH_PENALTY_ACCRUED: Accrued penalty fees exceed the monthly EMI amount.

3.2 Diagnosis Layer

The Diagnosis Layer parses user utterances to prevent redundant questioning. * **Purpose:** Compare user inputs against known parameters to figure out known facts, missing info, and the next logical question to ask. * **8 Intents Handled with Examples:** 1. EMI_INQUIRY: Inquiring about remaining EMIs or outstanding balance (e.g. *"How many EMIs are remaining on my loan?"*). 2. INTEREST_INQUIRY: Inquiring about principal vs interest payment splits (e.g. *"How much interest have I paid so far?"*). 3. PENALTY_INQUIRY: Inquiring about late charges (e.g. *"Why was a penalty charged on my account?"*). 4. PAYMENT_FAILURE: Checking transaction issues (e.g. *"My payment failed even though I had enough balance."*). 5. PENALTY_WAIVER: Requesting reversing of charges (e.g. *"Can you waive my penalty late charge since it was a bank error?"*). 6. SETTLEMENT_REQUEST: Requesting one-time settlement (e.g. *"I cannot pay my full loan, can I get a settlement?"*). 7. PAYMENT_COMMITMENT: Promising a future payment (e.g. *"I will pay my outstanding EMI next Tuesday."*). 8. GENERAL_INQUIRY: Standard fallbacks, greetings, or basic questions (e.g. *"Is anybody there?"* or *"Who are you?"*). * **Redundant Question Prevention:** Computes the set difference between required fields for the intent and known_facts. The agent only prompts the user for the remaining missing_info elements. * **Output:** DiagnosisResult object containing intent, confidence, known_facts, missing_info, next_question, can_resolve_immediately, suggested_tools, and resolution_hint.

3.3 RAG Engine

- **Keyword TF-IDF Approach:** Tokenizes and normalizes the search queries. It maps keywords against a local corpus, counting matches where title matches receive a 2x weight boost over body content matches.
- **20 Policy Documents Covered:** Contains 20 structured policy guidelines including:
 - Late payment charges calculation.

- Penalty waiver eligibility criteria.
- Autodebit (NACH) mandate failures.
- Foreclosure and pre-closure terms.
- CIBIL credit score reporting timelines.
- One-time settlement (OTS) approval guidelines.
- Human agent escalation protocols.
- Moratorium eligibility rules.
- **Prompt Injection:** Matches with a score above a similarity threshold are formatted as text and injected into the LLM system prompt as: [RELEVANT POLICY GUIDELINES: ...]
- **Limitations and Production Path:** Current keyword-based retrieval cannot handle semantic synonyms (e.g. "early payoff" does not match "foreclosure"). The production path replaces this with a vector database (e.g. ChromaDB, Pinecone) utilizing sentence-transformers embeddings (all-MiniLM-L6-v2) for semantic similarity search.

3.4 Memory Store

- **Two Memory Tables:**
 - `borrower_memory`: Stores user-specific attributes (e.g., preferences, commitments, callbacks).
 - `agent_memory`: Logs successful agent resolution paths.
- **5 Memory Types:**
 - **PREFERENCE:** Customer preferences such as preferred communication language, best time to call, or auto-debit channel.
 - **COMMITMENT:** Active promises to pay (contains amount, promised date, delay reason, and fulfillment status).
 - **FAQ:** Quick answers to customer questions about policies, avoiding repeated RAG retrieval for basic questions.
 - **RISK:** Chronic delinquency markers, default tendencies, or negative behavior profiles.
 - **CALLBACK:** Date, time, and reason for scheduled outbound calls.

- **Scenario 6 End-to-End:**

- **Call 1:** Borrower states: *"My salary got delayed, I will pay next Tuesday."* Agent parses intent, calculates the date, and saves the commitment: `{"amount": emi, "date": "2026-06-09", "fulfilled": false}`.
- **Call 2:** Borrower dials back 3 days later. The Context Engine loads the active commitment from the Memory Store. It overrides the default opening greeting to: *"Hi Arjun, during our last call you committed to paying ₹9,386.28 by June 9th. Were you able to make that payment?"*
- **Second Call Advantage:** By identifying past commitments instantly, the system avoids generic queries (like asking *"How can I help you?"*). It establishes rapid rapport, demonstrates organizational memory, and focuses the conversation directly on resolution.

3.5 LLM Agent

- **Model:** Groq Llama 3.1 (llama-3.1-8b-instant). Production environments upgrade to **Anthropic Claude 3.5 Sonnet** or **GPT-4** for enhanced tool orchestration.
- **System Prompt Construction:** Combined dynamically before invocation:
 - Unified Borrower Brief (CRM profile, LMS status, and recent Razorpay transactions).
 - Injected Memory (unfulfilled payment promises, callback schedules, customer preferences).
 - RAG Policy Guidelines (retrieved policies matching user intent).
 - Diagnosis Hint (intent, known facts, missing details, and next question suggestion).
- **Tool Use (8 Tools):**
 - `get_loan_details(borrower_id)`
 - `get_payment_history(borrower_id)`
 - `get_penalty_details(borrower_id)`
 - `create_ticket(borrower_id, category, subject, description)`
 - `generate_payment_link(borrower_id, amount)`
 - `schedule_callback(borrower_id, callback_date, reason)`
 - `escalate_to_human(borrower_id, reason, priority)`

- `search_knowledge_base(borrower_id, query)`
- **Mock Fallback Mode:** If Groq's API is rate-limited or offline, the system falls back to a deterministic rule-based template engine that parses intents using local regex matches and answers using local knowledge base snippets.

3.6 Voice Pipeline

- **Speech to Text (STT):** Deepgram nova-2 model, optimized for Indian English accents and noisy telephone lines.
 - **Text to Speech (TTS):** Mac system speech synthesis (`say -v Samantha` at 0.95 rate). Production version utilizes ElevenLabs.
 - **Data Flow:** `mic → WAV → Deepgram → text → agent → text → say`
-

4. Data Schema

The SQLite database (`data/borrowers.db`) contains the following 6 tables:

1. `borrowers` (22 columns)

Represents the unified customer loan contract ledger. * `borrower_id` (TEXT, PRIMARY KEY): Unique identifier. * `name` (TEXT): Full name of the borrower. * `phone` (TEXT): Primary contact phone number. * `email` (TEXT): Email address. * `city` (TEXT): Residence city. * `kyc_status` (TEXT): KYC status (PENDING, COMPLETED). * `pan_number` (TEXT): PAN details. * `bank_name` (TEXT): Linked bank name. * `loan_id` (TEXT): Unique loan ID. * `loan_type` (TEXT): Type (Home Loan, Personal Loan, etc.). * `loan_amount` (REAL): Principal amount. * `interest_rate` (REAL): Annual interest rate percentage. * `tenure_months` (INTEGER): Total tenure. * `emi_amount` (REAL): Monthly installment amount. * `loan_start_date` (TEXT): Start date. * `loan_end_date` (TEXT): Maturity date. * `outstanding_balance` (REAL): Outstanding principal balance. * `delinquency_status` (TEXT): Status (CURRENT, OVERDUE_15, OVERDUE_30, OVERDUE_60, NPA). * `overdue_days` (INTEGER): Overdue days count. * `penalty_amount` (REAL): Accrued penalty charges. * `call_count` (INTEGER): Number of calls made by borrower. * `sentiment_history` (TEXT): JSON array of past sentiment categories.

2. payments (11 columns)

Tracks payment transactions. * payment_id (TEXT, PRIMARY KEY): Unique identifier. * borrower_id (TEXT, FOREIGN KEY): Reference to borrower. * due_date (TEXT): Payment due date. * payment_date (TEXT): Actual payment date. * amount_due (REAL): Installment amount due. * amount_paid (REAL): Amount paid. * status (TEXT): Payment status (SUCCESS, FAILED, MISSED, AUTO_DEBIT_FAIL, PARTIAL). * failure_reason (TEXT): Reason for transaction failure. * gateway_ref (TEXT): Gateway reference ID. * channel (TEXT): Mode (UPI, NACH, DebitCard, etc.). * created_at (TEXT): Creation timestamp.

3. tickets (11 columns)

Stores service request logs. * ticket_id (TEXT, PRIMARY KEY): Unique identifier. * borrower_id (TEXT, FOREIGN KEY): Reference to borrower. * agent_id (TEXT): Assigned human support agent. * category (TEXT): Category (PENALTY_WAIVER, SETTLEMENT, etc.). * subject (TEXT): Brief subject. * description (TEXT): Details of query. * status (TEXT): Status (OPEN, RESOLVED, CLOSED). * priority (TEXT): Priority (LOW, MEDIUM, HIGH). * created_date (TEXT): Open date. * resolved_date (TEXT): Resolution date. * updated_at (TEXT): Last update timestamp.

4. conversations (10 columns)

Logs agent conversation sessions. * conversation_id (TEXT, PRIMARY KEY): Unique identifier. * borrower_id (TEXT, FOREIGN KEY): Reference to borrower. * session_id (TEXT): Associated websocket session ID. * transcript (TEXT): Full conversation transcript (JSON). * duration_seconds (INTEGER): Session duration. * rating (INTEGER): User feedback score. * notes (TEXT): Call summary notes. * created_at (TEXT): Open timestamp. * updated_at (TEXT): Close timestamp. * call_date (TEXT): Date of call.

5. borrower_memory (7 columns)

Stores persistent cross-call state variables. * memory_id (TEXT, PRIMARY KEY): Unique identifier. * borrower_id (TEXT, FOREIGN KEY): Reference to borrower. * memory_type (TEXT): Type (PREFERENCE, COMMITMENT, FAQ, RISK, CALLBACK). * key (TEXT): Lookup key (e.g. promise_to_pay). * value (TEXT): Details payload (JSON or plaintext). * created_at (TEXT): Creation timestamp. * updated_at (TEXT): Last update timestamp.

6. agent_memory (6 columns)

Tracks agent execution flows and successes. * memory_id (TEXT, PRIMARY KEY): Unique identifier. * borrower_id (TEXT, FOREIGN KEY): Reference to borrower. * memory_type (TEXT): Type (RESOLUTION_PATH). * key (TEXT): Key indicator (e.g., intent category). * value (TEXT): Executed tools and final status payload (JSON). * created_at (TEXT): Creation timestamp.

5. API Endpoints

Core Backend Routes (api/main.py)

- GET /: Returns API description, version, and details of available endpoints.
- POST /chat: REST endpoint to post conversation text and retrieve agent response (requires borrower_id, message, and session_id).
- GET /context/{identifier}: Context lookup returning structured data for a borrower by phone or ID.
- GET /memory/{borrower_id}: Direct memory lookup returning commitments, preferences, and scheduled callbacks.
- POST /memory/commitment: Manually registers a promise-to-pay commitment for a borrower.
- GET /health: Health status returning system timestamp, database presence, and knowledge base document count.

Dashboard Routes (api/dashboard.py)

- GET /: Serves the HTML dashboard UI.
- GET /metrics: Serves the system performance metrics page.
- GET /api/metrics: Dynamic aggregator returning delinquency breakdown, payment success rate, ticket resolution rate, intent distributions, and call volumes.
- GET /api/borrowers: Returns all borrowers for dashboard sidebar lists.
- GET /api/context/{phone}: Returns aggregated customer context information.

- GET /api/chat: Handles chat messages via GET parameters for fast dashboard updates.
-

6. Multi-System Integration

The system models integration with 5 independent enterprise platform classes within `core/system_integrations.py`:

1. **CRMSystem**: Simulates Zoho CRM. Stores core borrower profiles, name, phone, address, and KYC statuses.
2. **LoanManagementSystem (LMS)**: Simulates Finflux. Tracks loan tenures, EMIs, disbursals, interest rates, and overdue days.
3. **PaymentGatewaySystem**: Simulates Razorpay. Logs payment failures, payment channels, transaction IDs, and failure reasons.
4. **SupportTicketingSystem**: Simulates Freshdesk. Manages tickets for waivers and settlements.
5. **KnowledgeBaseSystem**: Simulates Confluence. Indexes policy files for quick lookup.

In production, each of these mock classes would be replaced with a real API client (using HTTP clients like `httplib` or SDKs) with no changes to the agent's core reasoning logic.

7. Demo Scenarios

The CredResolve voice agent is validated across 6 demo scenarios: 1. **Scenario 1: Remaining EMI Inquiry**: Borrower asks about outstanding tenure. The agent calls the LMS, computes remaining installments, and details the next EMI date and amount. 2. **Scenario 2: Interest Paid Inquiry**: Borrower asks for interest summaries. The agent calls Razorpay logs, splits principal vs interest paid, and calculates the outstanding balance. 3. **Scenario 3: Penalty Charge Inquiry**: Borrower queries late penalties. The agent checks overdue days, runs a RAG query on late policies, and explains the fee calculation. 4. **Scenario 4: Payment Failure**: Borrower complains about a failed transaction. The agent inspects the gateway failure code (e.g. bank-side glitch vs low balance) and suggests retry or waiver routes. 5. **Scenario 5: Penalty Waiver Request**: Borrower asks to waive late fees. The agent evaluates eligibility (e.g. good history + bank-side error), files a ticket in

Freshdesk, and explains timelines. 6. **Scenario 6: Follow-up with Memory:** Borrower calls back after a commitment. The agent loads the active promise-to-pay from the Memory Store and asks for an update.

8. Evaluation Coverage

The implementation covers all 7 criteria of the agentic evaluation framework:

Criteria	Weight	Implementation File	Status
Context Engine	20%	core/context_engine.py	✓ Complete
Diagnosis Layer	20%	core/diagnosis_layer.py	✓ Complete
Agentic Voice Experience	20%	agent/agent.py, agent/voice.py	✓ Complete
Multi-System Integration	15%	core/system_integrations.py	✓ Complete
RAG Implementation	10%	core/rag_engine.py	✓ Complete
Memory & Learning	10%	core/memory_store.py	✓ Complete
Production Readiness	5%	api/dashboard.py, api/main.py	✓ Complete

9. Known Limitations

- **LLM Model Limits:** The Groq free-tier model (llama-3.1-8b-instant) may encounter rate limits during heavy usage.
- **Basic TTS Quality:** Native Mac speech synthesis (say) lacks the natural intonation of production TTS systems.
- **Keyword RAG Limitations:** Keyword-based search cannot handle queries that lack exact term matches.
- **Mock Integrations:** Core APIs (Zoho CRM, Razorpay, Freshdesk) are simulated locally.
- **No Authentication:** The APIs lack JWT authentication or session validation.

- **Database Scaling:** SQLite is suitable for demos but does not scale to high concurrent write volumes.
-

10. Production Upgrade Path

To upgrade this system to production grade: * **LLM Brain:** Transition from Groq Llama 3 to **Anthropic Claude 3.5 Sonnet** (or GPT-4) with structured tool use definition for complex reasoning. * **TTS Pipeline:** Replace Mac TTS with **ElevenLabs Turbo v2** for natural voice output. * **Keyword RAG:** Index policy documents in a vector database like **ChromaDB** using **sentence-transformers** embeddings (all-MiniLM-L6-v2) for semantic search. * **Mock Systems:** Replace mock classes with real Zoho CRM, Finflux LMS, Razorpay Payment Gateway, and Freshdesk API clients. * **Database Scaling:** Migrate from SQLite to **PostgreSQL** (with pgvector for embedding searches). * **Security & Auth:** Secure all endpoints using **JWT** authentication on all endpoints.